詳解ホワイトボックススイッチNOS

MPLS Japan 2020

2020年11月9日

NTTエレクトロニクスアメリカ 石田 渉

アジェンダ

- ホワイトボックススイッチの近況
- ホワイトボックススイッチNOSのアーキテクチャ
- OSとEthernetASICの連携方法
- まとめ

ホワイトボックススイッチの近況

- ホワイトボックススイッチとそれ以外の境界が曖昧になってきた
  - Arista, Cisco, JuniperのハードでホワイトボックススイッチNOSのSONiCが動く
  - Cisco ASIC Silicon OneのプレスリリースでSONiC/SAIが言及される
  - 一方ホワイトボックススイッチ+OSS NOSの商用サポート提供も進んでいる
  - Cumulus, MellanoxはNVIDIAに買収される

- データセンタネットワーク以外にも適用しようとする動きが広がっている
  - DCSG(セルサイトゲートウェイ) - TIP, AT&T
  - シャーシ型ルータ - OCP, AT&T
  - 伝送装置 - TIP, ONF ODTN
  - OLT - ONF VOLTHA
  - Enterprise Edge : DENT (Amazon, NVIDIA etc..)

ホワイトボックススイッチNOSの近況

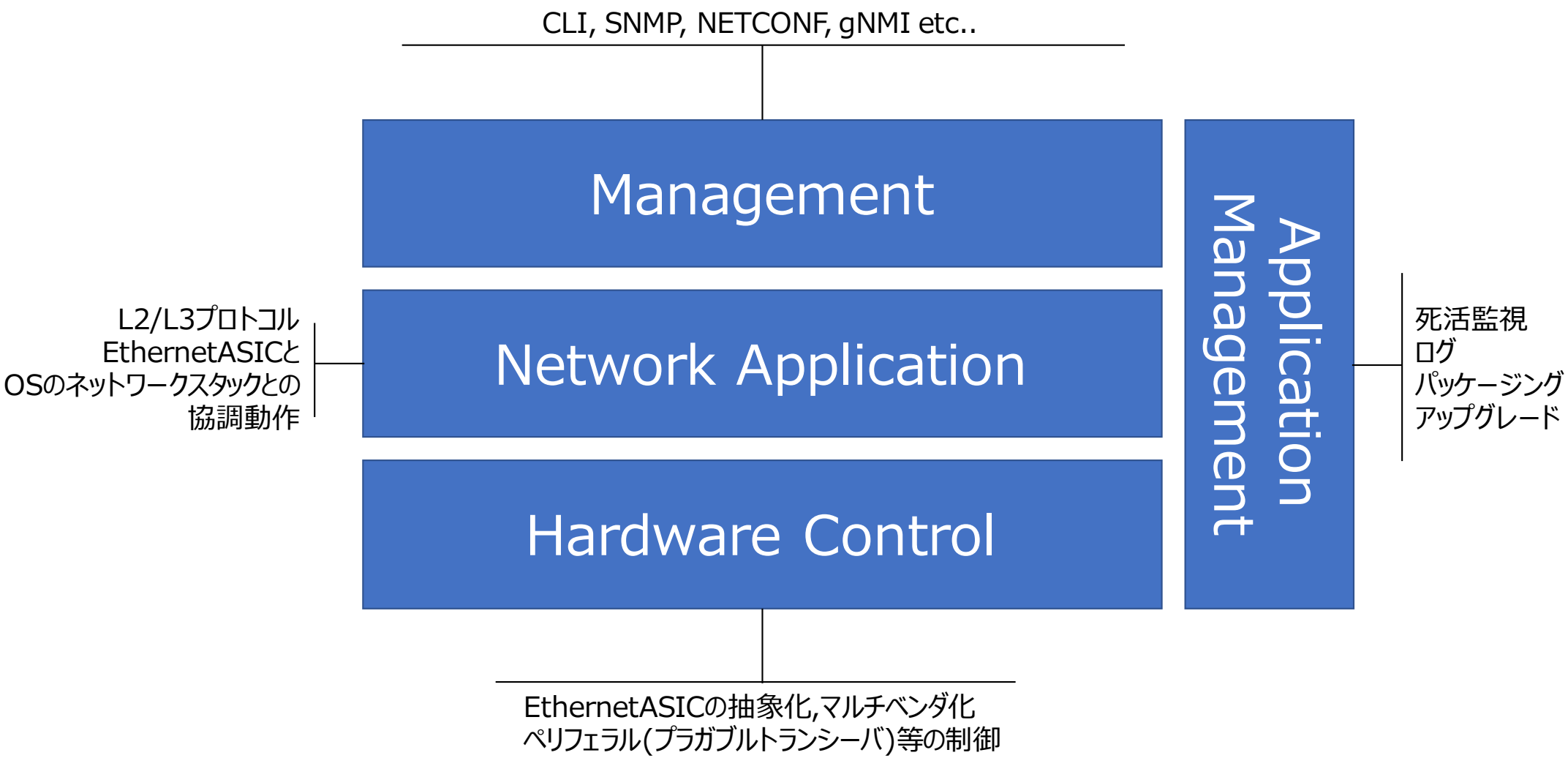
- OSS: SONiC, DANOS, Stratum, DENT

- SONiCの開発が活発 / OSSの中での趨勢は決まった？
  - Microsoft, Broadcom, Dell, NVIDIA(Mellanox)等が開発に参加
  - とはいえ誰でも簡単に安心して使えるレベルにはまだまだ達していない
  - IPInfusion, Edgecore等が商用サポートを開始

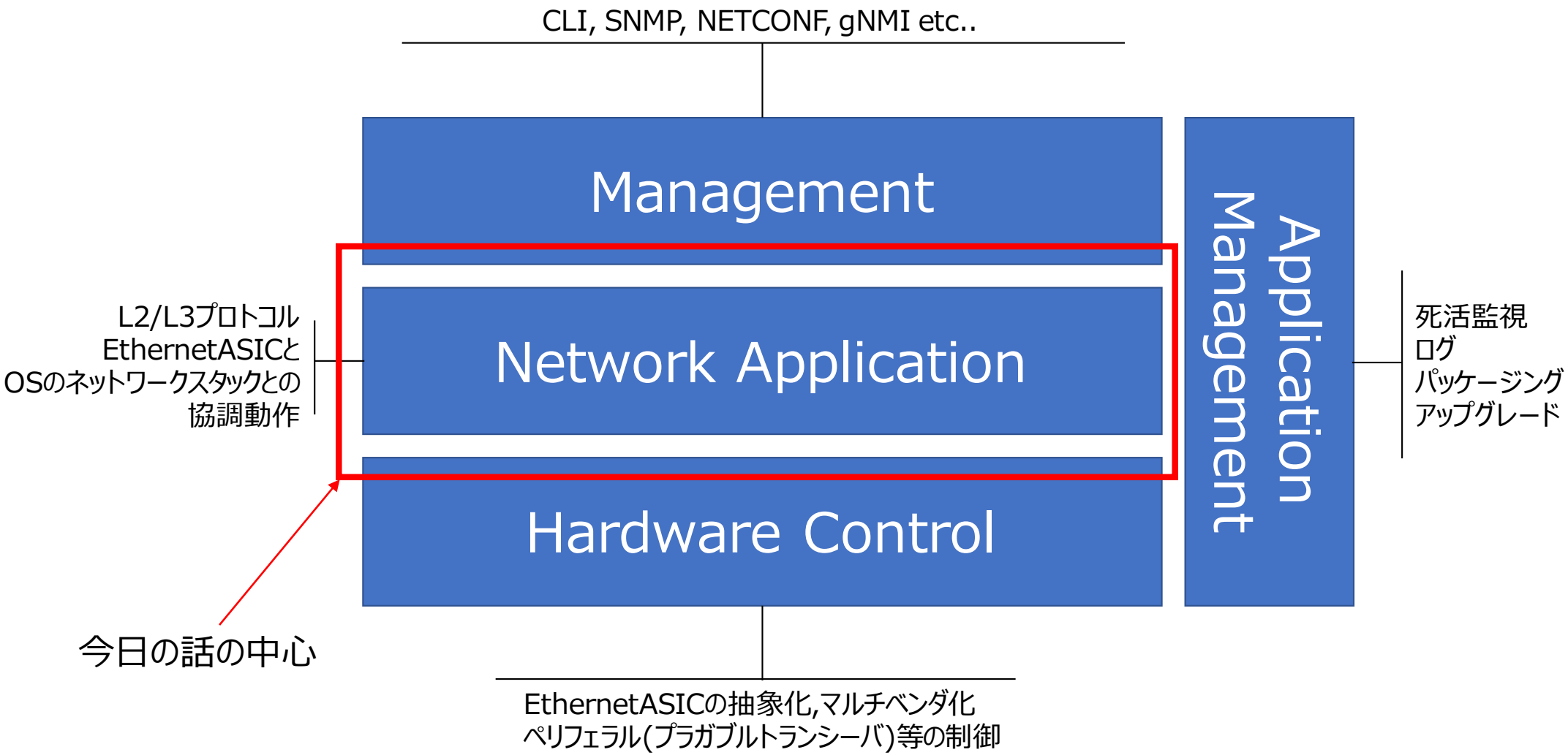
- FBOSS(Facebook)がSAIの利用を開始

- ONL(Open Network Linux)が多くのNOSで使われながらもBigSwitchがAristaに買収されて先行きが不透明
  - FBOSS, ArcOS, DENT, StractumはONLベース

ホワイトボックススイッチNOSのアーキテクチャ



ホワイトボックススイッチNOSのアーキテクチャ



Hardware Control

EthernetASICの抽象化

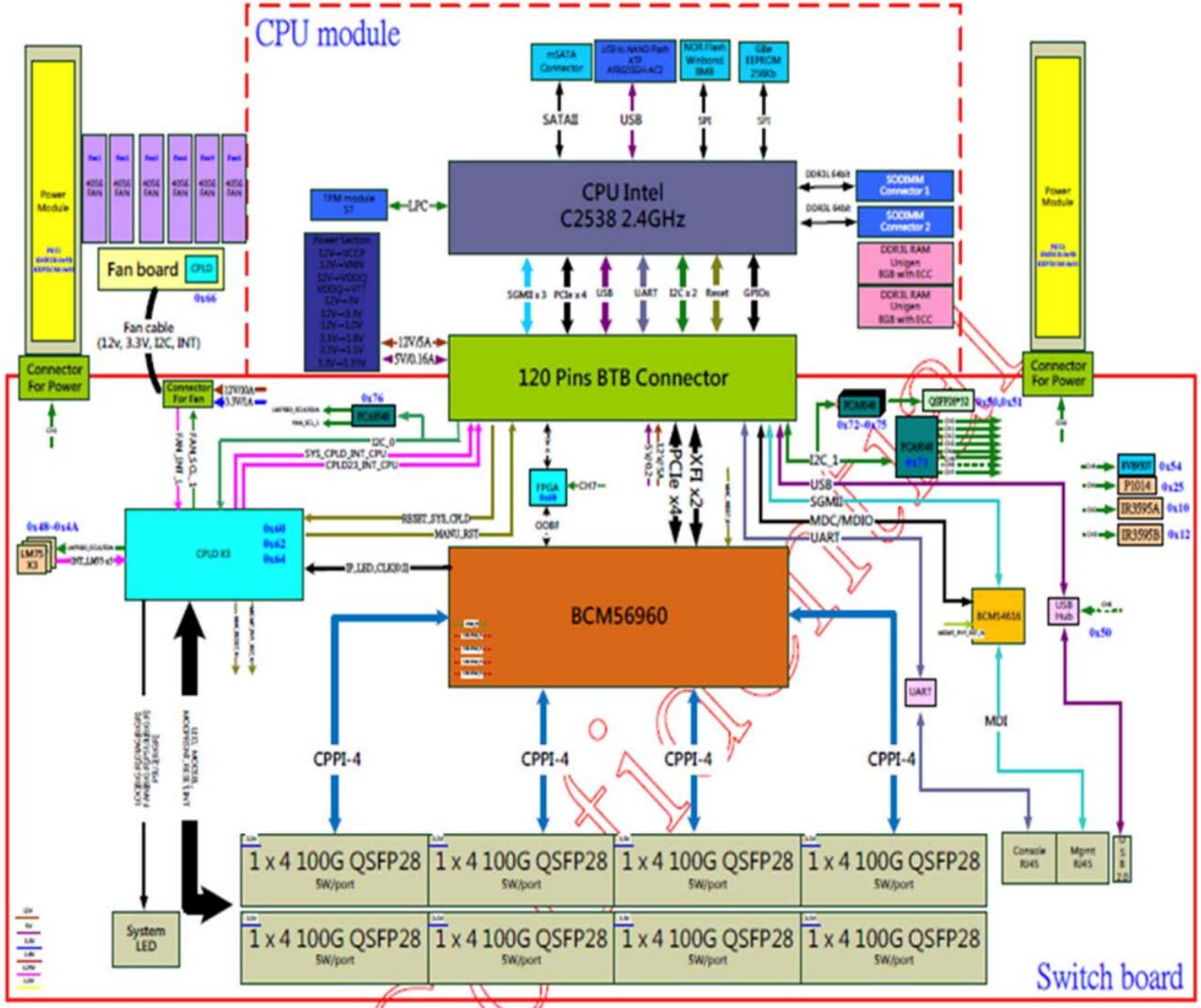
- OSからはPCIeデバイスとして見える
- 制御はチップベンダが提供するユーザランドで動くSDKで行うのが一般的
- NOSは複数ベンダのEthernetASICの制御を抽象化するため、HAL(Hardware Abstraction Layer)をベンダのSDK上に載せることが一般的
- SONiCではSAI(Switch Abstraction Interface)というオープンなHALを利用している
  - Broadcom/NVIDIA(Mellanox)を始め多くのEthernetASICが対応しており、オープンソースのHALでは一番成功している
    - Cisco Silicon ONEもSAIへの対応を発表している
  - 最近ではFBOSSもSAIをHALとして利用、今後デファクトスタンダードになっていく？

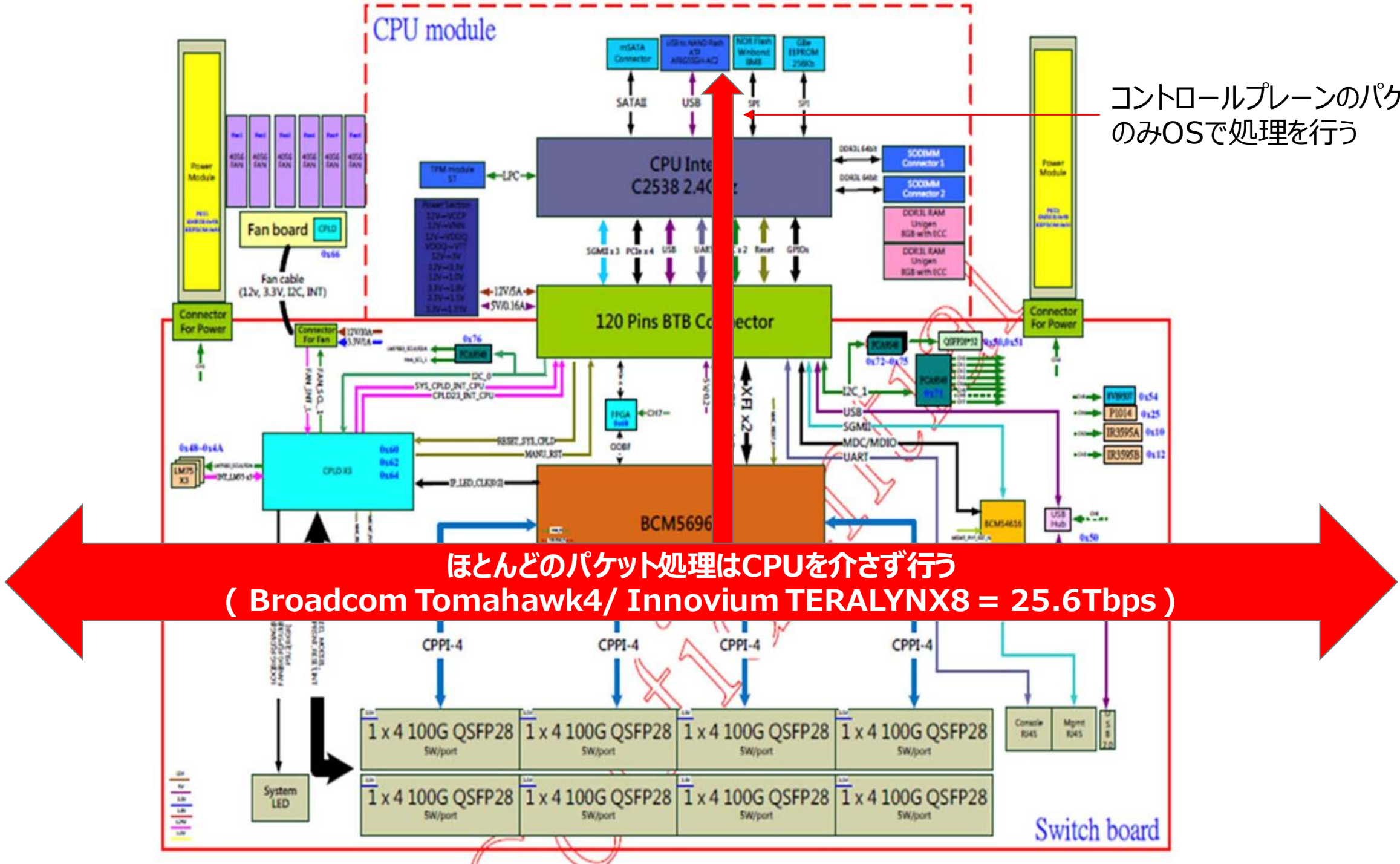
Hardware Control

ペリフェラルの制御

- ネットワーク機器はサーバ寄りではあるが、組み込み機器の要素もある
  - BMC/IPMIの利用も広がっているが、まだ温度モニタリング, ファン制御はOS上で行わなければいけないことが一般的
  - トランシーバの制御
    - サーバではethtoolを使いNICのドライバがPCIeを経由してトランシーバにアクセス
    - ホワイトボックススイッチではトランシーバのi2cバスが直接CPUに接続されているため、接続方法が異なる
      - ethtoolはそのまま使えない

- ピザボックス型だけではなく、シャーシ型/モジュール型のホワイトボックス製品も増えており、ペリフェラルを統一して扱える仕組みが必要とされている
  - Cumulusが提唱したACPIベースのAPDは支持が得られなかった様子
  - ONLのONLPが現状有力解だがSONiCが使っていない + ONLの今後の開発動向が不透明


CPU module
mSATA Connector
USB to NAND Flash
NOR Flash Winbond 8MB
GBe EEPROM 256Kb
SATAII
USB
SPI
SPI
CPU Intel C2538 2.4GHz
TPM module ST
LPC
DDR3L 64bit
DDR3L 64bit
SODIMM Connector 1
SODIMM Connector 2
DDR3L RAM Unigen 8GB with ECC
DDR3L RAM Unigen 8GB with ECC
SGMII x 3
PCIe x 4
USB
UART
I2C x 2
Reset
GPIOs
120 Pins BTB Connector
Power Module
Fan board
CPLD
0x66
Fan cable (12v, 3.3V, I2C, INT)
Connector For Power
Connector For Fan
12V/0.5A
5V/0.16A
0x76
PCA9548
0x72~0x75
QSFP28*32
0x50,0x51
I2C_0
SYS_CPLD_INT_CPU
CPLD23_INT_CPU
FPGA
CH7
ODBF
PCIe x4
XFI x2
I2C_1
USB
SGMII
MDC/MDIO
UART
0x48~0x4A
LM75 X3
CPLD X3
0x60
0x62
0x64
RESET_SYS_CPLD
MANU_RST
P_LED_CLK&DT
BCM56960
BCM54616
UART
MDI
USB Hub
0x50
IVB950T
0x54
PI3014
0x25
IR3595A
0x10
IR3595B
0x12
CPPI-4
CPPI-4
CPPI-4
CPPI-4
1 x 4 100G QSFP28
5W/port
1 x 4 100G QSFP28
5W/port
1 x 4 100G QSFP28
5W/port
1 x 4 100G QSFP28
5W/port
1 x 4 100G QSFP28
5W/port
1 x 4 100G QSFP28
5W/port
1 x 4 100G QSFP28
5W/port
1 x 4 100G QSFP28
5W/port
Console RJ45
Mgmt RJ45
USB 2.0
System LED
Switch board



コントロールプレーンのパケット
のみOSで処理を行う

ほとんどのパケット処理はCPUを介さず行う
(Broadcom Tomahawk4/ Innovium TERALYNX8 = 25.6Tbps)

Hardware Control

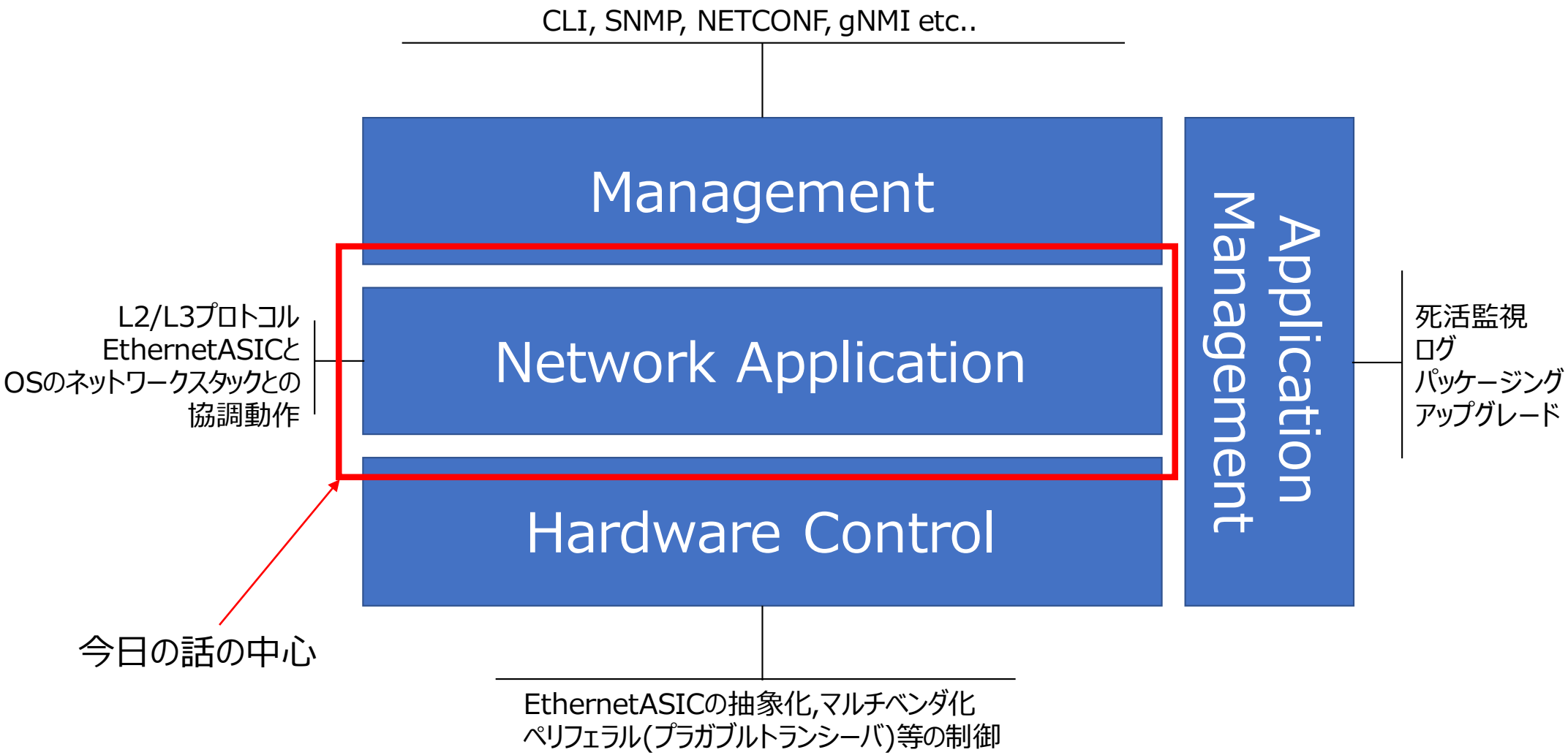
SAI (Switch Abstraction Interface)

- オープンなEthernetASICの制御API
- https://github.com/opencomputeproject/SAI

- Cのヘッダファイルの集合 + ユーティリティライブラリ
  - CRUDベースのAPI ：オブジェクトの作成/削除 + アトリビュートの読み書き
  - notification/callbackもアトリビュートとして関数ポインタを登録するようになっている

- SAIの実装は各EthernetASICベンダが行い、OSS化する必要はない
  - Barefoot, Broadcom, Cavium, Centec, Innovium, Marvell, Mellanox, Nephosが実装を公表

ホワイトボックススイッチNOSのアーキテクチャ



Network Application

- NOSではEthernetASICとOSのネットワークスタックをどのように連携させるかが肝
- なぜ連携させる必要があるのか？
  - サーバサイドのソフト資産をそのまま流用したいから
  - 流用しないとTCPスタックに始まり、ping, tcpdump, ルーティングデーモン等々全てに修正が必要になる
- スイッチ上で適当なLinuxディストリビューションを立ち上げればマネジメントポートは普通のネットワークインターフェイスとして見え、サーバと同じように管理できる
- しかしEthernetASICのフロントパネルポートはそのままでは見えない
  - ip link showでフロントパネルポートは見えない
- またEthernetASICの状態とOSのネットワークスタックを同期させないといけない
  - FDB(L2), FIB(L3)等々

OSのネットワークスタックとの連携例

1. EthernetASICのフロントパネルポートに対応するネットワークインターフェイスの作成
2. インターフェイスへのIPアドレスの設定
3. Neighborテーブル(ARP/ND)の同期
4. ルーティングプロトコルとの連携

これらをSONiC/SAIではどのように実現しているかをSAIに対してどのようなAPIを発行するか、を中心に見ていく

1. ネットワークインターフェイスの作成

- SAIではhostifというオブジェクトがOS内のインターフェイスに対応する
  - hostifを作ると、SAIライブラリがOS内にnetdevを生やしてくれる

```
19  *
20  * @file    saihostif.h
21  *
22  * @brief   This module defines SAI host interface
23  *
24  * @par Abstract
25  *
26  *    This module defines SAI Host Interface which is responsible for
27  *    creating/deleting Linux netdev corresponding to the host interface type.
28  *    All the management operations of the netdevs such as changing IP address
29  *    are outside the scope of SAI.
30  */
31
```

https://github.com/opencomputeproject/SAI/blob/master/inc/saihostif.h#L20-L30

Host Interfaceの作成

```
1342  /**
1343   * @brief Hostif methods table retrieved with sai_api_query()
1344   */
1345  typedef struct _sai_hostif_api_t
1346  {
1347      sai_create_hostif_fn                        create_hostif;
1348      sai_remove_hostif_fn                        remove_hostif;
1349      sai_set_hostif_attribute_fn                 set_hostif_attribute;
1350      sai_get_hostif_attribute_fn                 get_hostif_attribute;
1351      sai_create_hostif_table_entry_fn            create_h
1352      sai_remove_hostif_table_entry_fn            remove_h
1353      sai_set_hostif_table_entry_attribute_fn     set_host
1354      sai_get_hostif_table_entry_attribute_fn     get_host
1355      sai_create_hostif_trap_group_fn             create_h
1356      sai_remove_hostif_trap_group_fn             remove_h
1357      sai_set_hostif_trap_group_attribute_fn      set_host
1358      sai_get_hostif_trap_group_attribute_fn      get_host
1359      sai_create_hostif_trap_fn                   create_h
1360      sai_remove_hostif_trap_fn                   remove_h
1361      sai_set_hostif_trap_attribute_fn            set_host
1362      sai_get_hostif_trap_attribute_fn            get_host
1363      sai_create_hostif_user_defined_trap_fn      create_h
1364      sai_remove_hostif_user_defined_trap_fn      remove_h
1365      sai_set_hostif_user_defined_trap_attribute_fn set_host
1366      sai_get_hostif_user_defined_trap_attribute_fn get_hostif_user_defined_trap_attribute;
1367      sai_recv_hostif_packet_fn                   recv_hostif_packet;
1368      sai_send_hostif_packet_fn                   send_hostif_packet;
1369      sai_allocate_hostif_packet_fn               allocate_hostif_packet;
1370      sai_free_hostif_packet_fn                   free_hostif_packet;
1371  } sai_hostif_api_t;
1372
```

```
886  /**
887   * @brief Create host interface
888   *
889   * @param[out] hostif_id Host interface id
890   * @param[in] switch_id Switch object id
891   * @param[in] attr_count Number of attributes
892   * @param[in] attr_list Array of attributes
893   *
894   * @return #SAI_STATUS_SUCCESS on success, failure status code on error
895   */
896  typedef sai_status_t (*sai_create_hostif_fn)(
897          _Out_ sai_object_id_t *hostif_id,
898          _In_ sai_object_id_t switch_id,
899          _In_ uint32_t attr_count,
900          _In_ const sai_attribute_t *attr_list);
901
```

スイッチIDとアトリビュートのリストを受け取り、OIDを返す
(スイッチID = ASICの通し番号, ピザボックスタイプのスイッチでは
通常ASICは1つのみ搭載なのでスイッチIDは常に同じ)

https://github.com/opencomputeproject/SAI/blob/master/inc/saihostif.h#L1347

```
785 /**
786  * @brief Host interface attribute IDs
787  */
788 typedef enum _sai_hostif_attr_t
789 {
790     /**
791      * @brief Start of attributes
792      */
793     SAI_HOSTIF_ATTR_START,
794
795     /**
796      * @brief Host interface type
797      *
798      * @type sai_hostif_type_t
799      * @flags MANDATORY_ON_CREATE | CREATE_ONLY
800      */
801     SAI_HOSTIF_ATTR_TYPE = SAI_HOSTIF_ATTR_START,
802
803     /**
804      * @brief Host interface object ID
805      *
806      * Port netdev will be created when object type is SAI_OBJECT_TYPE_PORT
807      * LAG netdev will be created when object type is SAI_OBJECT_TYPE_LAG
808      * VLAN netdev will be created when object type is SAI_OBJECT_TYPE_VLAN
809      * System Port netdev will be created when object type is SAI_OBJECT_TYPE_SYSTE
810      *
811      * @type sai_object_id_t
812      * @flags MANDATORY_ON_CREATE | CREATE_ONLY
813      * @objects SAI_OBJECT_TYPE_PORT, SAI_OBJECT_TYPE_LAG, SAI_OBJECT_TYPE_VLAN, SA
814      * @condition SAI_HOSTIF_ATTR_TYPE == SAI_HOSTIF_TYPE_NETDEV
815      */
816     SAI_HOSTIF_ATTR_OBJ_ID,
```

Host Interfaceのアトリビュートの定義

コメントにアトリビュートの型とどのような使いかたが出来るかを示すフラグ等が記載されている
SONiCではこれらを機械的にパースして制約を満たしているかチェックする仕組みを利用している(metaライブラリ)

```
737
738 /**
739  * @brief Attribute data for SAI_HOSTIF_ATTR_TYPE
740  */
741 typedef enum _sai_hostif_type_t
742 {
743     /** Netdevice */
744     SAI_HOSTIF_TYPE_NETDEV,
745
746     /** File descriptor */
747     SAI_HOSTIF_TYPE_FD,
748
749     /** Generic netlink */
750     SAI_HOSTIF_TYPE_GENETLINK
751
752 } sai_hostif_type_t;
753
```

OSにインターフェイスを生やすにはNETDEVを選択

Host Interfaceのアトリビュート(2/2)

```
785  /**
786   * @brief Host interface attribute IDs
787   */
788  typedef enum _sai_hostif_attr_t
789  {
790      /**
791       * @brief Start of attributes
792       */
793      SAI_HOSTIF_ATTR_START,
794
795      /**
796       * @brief Host interface type
797       *
798       * @type sai_hostif_type_t
799       * @flags MANDATORY_ON_CREATE | CREATE_ONLY
800       */
801      SAI_HOSTIF_ATTR_TYPE = SAI_HOSTIF_ATTR_START,
802
803      /**
804       * @brief Host interface object ID
805       *
806       * Port netdev will be created when object type is SAI_OBJECT_TYPE_PORT
807       * LAG netdev will be created when object type is SAI_OBJECT_TYPE_LAG
808       * VLAN netdev will be created when object type is SAI_OBJECT_TYPE_VLAN
809       * System Port netdev will be created when object type is SAI_OBJECT_TYPE_SYSTEM_PORT
810       *
811       * @type sai_object_id_t
812       * @flags MANDATORY_ON_CREATE | CREATE_ONLY
813       * @objects SAI_OBJECT_TYPE_PORT, SAI_OBJECT_TYPE_LAG, SAI_OBJECT_TYPE_VLAN, SAI_OBJECT_TYPE_SYSTEM_PORT
814       * @condition SAI_HOSTIF_ATTR_TYPE == SAI_HOSTIF_TYPE_NETDEV
815       */
816      SAI_HOSTIF_ATTR_OBJ_ID,
817
```

https://github.com/opencomputeproject/SAI/blob/master/inc/saihostif.h#L816

Hostifを作るには、対応するPort(フロントパネルポート)のOIDを指定することが必須であることがわかる

HostifはPort以外にもLAG, VLAN, SYSTEM_PORT(シャーシ型の装置で利用)に対して作成することもできる。Portと同様対応するOIDを指定する。

またフラグにより、このアトリビュートは作成時には必須かつ作成時にのみ利用できる(後から変更できない)、とわかる。

Portの作成

Hostifを作るためには、事前にPortを作らなければいけない
Portの作成にはcreate_port APIを利用する

```
3078  /**
3079   * @brief Port methods table retrieved with sai_api_query()
3080   */
3081  typedef struct _sai_port_api_t
3082  {
3083      sai_create_port_fn                  create_port;
3084      sai_remove_port_fn                  remove_port;
3085      sai_set_port_attribute_fn           set_port_attribute;
3086      sai_get_port_attribute_fn           get_port_attribute;
3087      sai_get_port_stats_fn               get_port_stats;
3088      sai_get_port_stats_ext_fn           get_port_stats_ex
3089      sai_clear_port_stats_fn             clear_port_stats;
3090      sai_clear_port_all_stats_fn         clear_port_all_st
3091      sai_create_port_pool_fn             create_port_pool;
3092      sai_remove_port_pool_fn             remove_port_pool;
3093      sai_set_port_pool_attribute_fn      set_port_pool_att
3094      sai_get_port_pool_attribute_fn      get_port_pool_att
3095      sai_get_port_pool_stats_fn          get_port_pool_sta
3096      sai_get_port_pool_stats_ext_fn      get_port_pool_sta
3097      sai_clear_port_pool_stats_fn        clear_port_pool_s
3098      sai_create_port_connector_fn        create_port_conne
3099      sai_remove_port_connector_fn        remove_port_conne
3100      sai_set_port_connector_attribute_fn set_port_connecto
3101      sai_get_port_connector_attribute_fn get_port_connecto
3102      sai_create_port_serdes_fn           create_port_serdes;
3103      sai_remove_port_serdes_fn           remove_port_serdes;
3104      sai_set_port_serdes_attribute_fn    set_port_serdes_attribute;
3105      sai_get_port_serdes_attribute_fn    get_port_serdes_attribute;
3106  } sai_port_api_t;
3107
```

```
2370  /**
2371   * @brief Create port
2372   *
2373   * @param[out] port_id Port id
2374   * @param[in] switch_id Switch id
2375   * @param[in] attr_count Number of attributes
2376   * @param[in] attr_list Array of attributes
2377   *
2378   * @return #SAI_STATUS_SUCCESS on success, failure status code on error
2379   */
2380  typedef sai_status_t (*sai_create_port_fn)(
2381          _Out_ sai_object_id_t *port_id,
2382          _In_ sai_object_id_t switch_id,
2383          _In_ uint32_t attr_count,
2384          _In_ const sai_attribute_t *attr_list);
2385
```

スイッチIDとアトリビュートのリストを受け取り、OIDを返す

https://github.com/opencomputeproject/SAI/blob/master/inc/saiport.h#L3083

master ▾ **sonic-buildimage** / device / arista / x86_64-arista_7260cx3_64 / Arista-7260CX3-C64 / **port_config.ini**

yxieca [Arista7260CX3] add port speed information to port_config.ini (#1412) ...

2 contributors

67 lines (67 sloc) | 3.99 KB

	# name	lanes	alias	index	speed
2	Ethernet0	77,78,79,80	Ethernet1/1	1	100000
3	Ethernet4	65,66,67,68	Ethernet2/1	2	100000
4	Ethernet8	85,86,87,88	Ethernet3/1	3	100000
5	Ethernet12	89,90,91,92	Ethernet4/1	4	100000

```
618     /**
619      * @brief Hardware Lane list
620      *
621      * @type sai_u32_list_t
622      * @flags MANDATORY_ON_CREATE | CREATE_ONLY | KEY
623      */
624     SAI_PORT_ATTR_HW_LANE_LIST,
625
626     /**
627      * @brief Speed in Mbps
628      *
629      * On get, returns the configured port speed.
630      *
631      * @type sai_uint32_t
632      * @flags MANDATORY_ON_CREATE | CREATE_AND_SET
633      */
634     SAI_PORT_ATTR_SPEED,
```

Portの作成ではHW_LANE_LISTとSPEEDを必ず設定しなければいけない

HW_LANE_LISTにはプラットフォーム固有のSERDESレーン番号のリストを渡す
QSFP28/100GbEの場合、電気レーンは4本なので4要素のリストになる

SONiCではport_config.iniというプラットフォーム固有のファイルにレーン番号の情報が格納される (ただしdynamic port breakoutのサポートが進んでおり、これが完了するとport_config.iniは使われなくなる予定.
https://github.com/Azure/SONiC/blob/master/doc/dynamic-port-breakout/sonic-dynamic-port-breakout-HLD.md)

OSのネットワークスタックとの連携

1. EthernetASICのフロントパネルポートに対応するネットワークインターフェイスの作成
   - PortとHost Interfaceオブジェクトを作成する
2. インターフェイスへのIPアドレスの設定
3. Neighborテーブル(ARP/ND)の同期
4. ルーティングプロトコルとの連携

2. インターフェイスへのIPアドレスの設定

- LinuxではnetlinkというAPIを使って、インターフェイスへのIPアドレスを設定する (ipコマンドはnetlinkを内部で利用している)
  - ip address add 192.168.0.1/24 dev Ethernet1 ← 1.で作成したnetdev
- しかしkernelはEthernetASICの制御に関与しないので、上記コマンドではソフト(OS)にはIPアドレスを設定できても、ハード(EthernetASIC)にはIPアドレスが設定できていない
- EthernetASICに対しては自身のIPアドレスをCPU側に転送するように設定する必要がある
- LinuxのネットワークスタックとEthernetASICの制御をどちらともアトミックに行う必要がある

OS/EthernetASIC両者への操作の実現方法

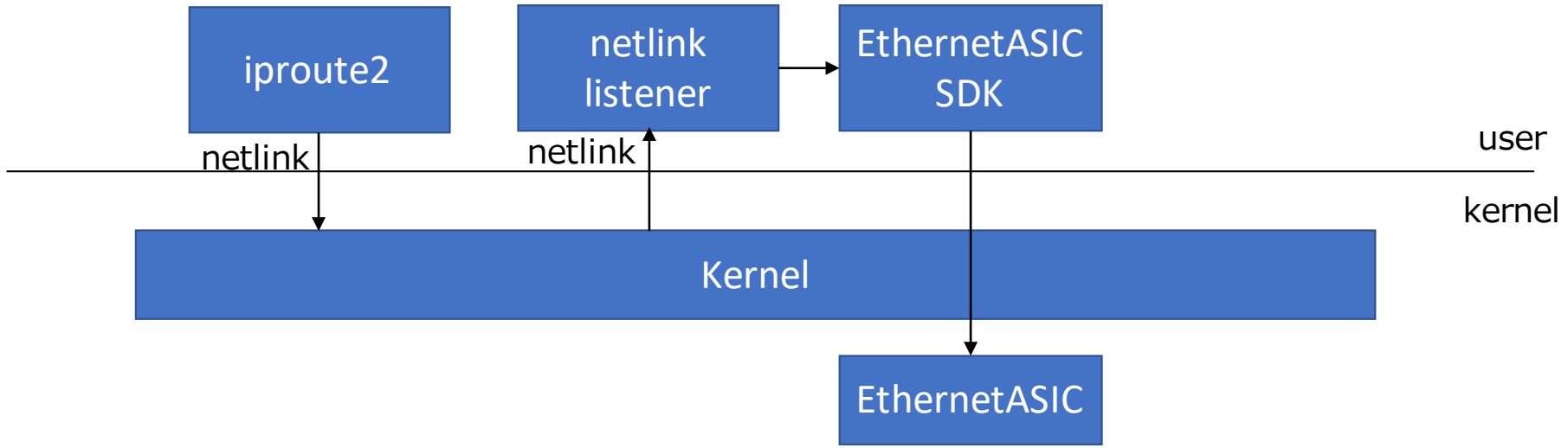
1. 普通のLinuxサーバと同じように操作出来るようにする

2. 専用CLI/マネジメントシステムを用意する

OS/EthernetASIC両者への操作の実現方法

1. 普通のLinuxサーバと同じように操作出来るようにする

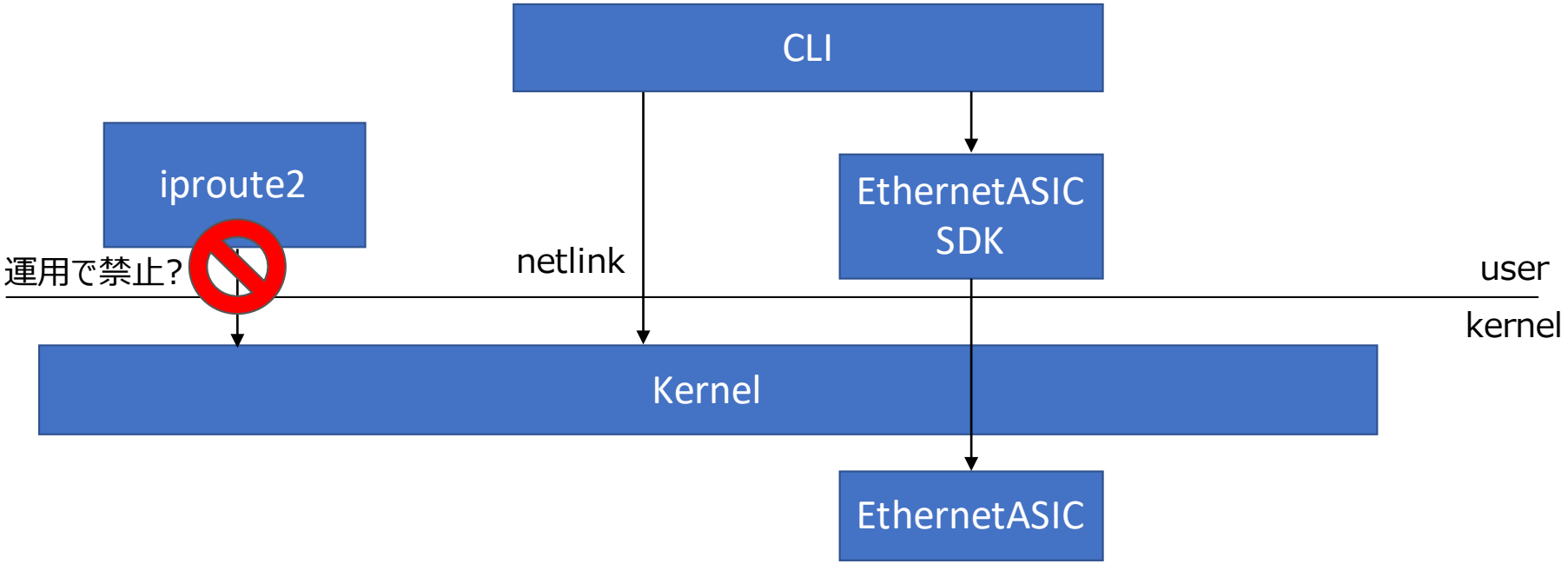
- netlinkはユーザ空間からモニタリングすることができる（ip monitor）
- ユーザのipコマンドによる変更をnetlink listenerで検知して、その内容を元にEthernetASIC SDKを介して、EthernetASICを制御する
- ipコマンドの変更はKernelへの処理が完了したら成功してしまうので、EthernetASIC SDKがエラーを返したときのエラーハンドリングができない



OS/EthernetASIC両者への操作の実現方法

2. 専用CLI/マネジメントシステムを用意する

- Kernelの設定とEthernetASICの設定どちらとも成功してはじめて、ユーザに成功した、と返すCLIを用意する
- ユーザがNET_ADMIN権限がある場合、簡単にシステムを壊せてしまう
  - ip address addコマンドのユーザによる実行でKernelとEthenetASICの状態が不一致
- SONiCはこの方式



インターフェイスへのIPアドレスの設定

EthernetASICに対して必要な操作

1. routerinterfaceの作成
   - 主な用途
     1. directly connectedな経路のnexthopとして
     2. それ以外の経路のnexthopが使うインターフェイスとして

2. L3経路の作成
   1. 自分自身への経路作成
      - nexthopはCPUに設定
   2. directly connectedな経路の作成
      - nexthopは上で作成したrouterinterfaceに設定

Router Interfaceの作成

```
/**
 * @brief Routing interface methods table retrieved with sai_api_query()
 */
typedef struct _sai_router_interface_api_t
{
    sai_create_router_interface_fn          create_router_interface;
    sai_remove_router_interface_fn          remove_router_interface;
    sai_set_router_interface_attribute_fn   set_router_interface_attribute;
    sai_get_router_interface_attribute_fn   get_router_interface_attribute;
    sai_get_router_interface_stats_fn       get_router_interface_stats;
    sai_get_router_interface_stats_ext_fn   get_router_interface_stats_ext;
    sai_clear_router_interface_stats_fn     clear_router_interface_stats;

} sai_router_interface_api_t;
```

```
/**
 * @brief Create router interface.
 *
 * @param[out] router_interface_id Router interface id
 * @param[in] switch_id Switch id
 * @param[in] attr_count Number of attributes
 * @param[in] attr_list Array of attributes
 *
 * @return #SAI_STATUS_SUCCESS on success, failure status code on error
 */
typedef sai_status_t (*sai_create_router_interface_fn)(
        _Out_ sai_object_id_t *router_interface_id,
        _In_ sai_object_id_t switch_id,
        _In_ uint32_t attr_count,
        _In_ const sai_attribute_t *attr_list);
```

スイッチIDとアトリビュートのリストを受け取り、OIDを返す

https://github.com/opencomputeproject/SAI/blob/master/inc/sairouterinterface.h#L437

Router Interfaceのアトリビュート

```
64  /**
65   * @brief Routing interface attribute IDs
66   */
67  typedef enum _sai_router_interface_attr_t
68  {
69      /**
70       * @brief Start of attributes
71       */
72      SAI_ROUTER_INTERFACE_ATTR_START,
73
74      /* READ-ONLY */
75
76      /**
77       * @brief Virtual router id
78       *
79       * @type sai_object_id_t
80       * @flags MANDATORY_ON_CREATE | CREATE_ONLY
81       * @objects SAI_OBJECT_TYPE_VIRTUAL_ROUTER
82       */
83      SAI_ROUTER_INTERFACE_ATTR_VIRTUAL_ROUTER_ID = SAI_ROUTER_INTERFACE_ATTR_START,
84
85      /**
86       * @brief Router interface type
87       *
88       * @type sai_router_interface_type_t
89       * @flags MANDATORY_ON_CREATE | CREATE_ONLY
90       */
91      SAI_ROUTER_INTERFACE_ATTR_TYPE,
92
93      /**
94       * @brief Associated Port, System Port or LAG object id
95       *
96       * @type sai_object_id_t
97       * @flags MANDATORY_ON_CREATE | CREATE_ONLY
98       * @objects SAI_OBJECT_TYPE_PORT, SAI_OBJECT_TYPE_LAG, SAI_OBJECT_TYPE_SYSTEM_PORT
99       * @condition SAI_ROUTER_INTERFACE_ATTR_TYPE == SAI_ROUTER_INTERFACE_TYPE_PORT or SAI_ROUTER_INTERFACE_ATTR_TYPE == SAI_ROUTER_INTERFACE_TYPE_SUB_PORT
100      */
101     SAI_ROUTER_INTERFACE_ATTR_PORT_ID,
```

ルーティングインスタンス(VR)の指定
(VRFの実装で利用)

対応するポートのOIDを指定

```
36  /**
37   * @brief Attribute data for #SAI_ROUTER_INTERFACE_ATTR_TYPE
38   */
39  typedef enum _sai_router_interface_type_t
40  {
41      /** Port or LAG or System Port Router Interface Type */
42      SAI_ROUTER_INTERFACE_TYPE_PORT,
43
44      /** VLAN Router Interface Type */
45      SAI_ROUTER_INTERFACE_TYPE_VLAN,
46
47      /** Loopback Router Interface Type */
48      SAI_ROUTER_INTERFACE_TYPE_LOOPBACK,
49
50      /** MPLS Router Interface Type */
51      SAI_ROUTER_INTERFACE_TYPE_MPLS_ROUTER,
52
53      /** Sub port Router Interface Type */
54      SAI_ROUTER_INTERFACE_TYPE_SUB_PORT,
55
56      /** .1D Bridge Router Interface Type */
57      SAI_ROUTER_INTERFACE_TYPE_BRIDGE,
58
59      /** Q-in-Q Router Interface Type */
60      SAI_ROUTER_INTERFACE_TYPE_QINQ_PORT,
61
62  } sai_router_interface_type_t;
```

https://github.com/opencomputeproject/SAI/blob/master/inc/sairouterinterface.h#L42

Routeの作成

```
316  /**
317   * @brief Router entry methods table retrieved with sai_api_query()
318   */
319  typedef struct _sai_route_api_t
320  {
321      sai_create_route_entry_fn                   create_route_entry;
322      sai_remove_route_entry_fn                   remove_route_entry;
323      sai_set_route_entry_attribute_fn            set_route_entry_attribute;
324      sai_get_route_entry_attribute_fn            get_route_entry_attribute;
325
326      sai_bulk_create_route_entry_fn              create_route_entries;
327      sai_bulk_remove_route_entry_fn              remove_route_entries;
328      sai_bulk_set_route_entry_attribute_fn       set_route_entries_attribute;
329      sai_bulk_get_route_entry_attribute_fn       get_route_entries_attribute;
330
331  } sai_route_api_t;
```

```
170  /**
171   * @brief Create Route
172   *
173   * Note: IP prefix/mask expected in Network Byte Order.
174   *
175   * @param[in] route_entry Route entry
176   * @param[in] attr_count Number of attributes
177   * @param[in] attr_list Array of attributes
178   *
179   * @return #SAI_STATUS_SUCCESS on success, failure status code on error
180   */
181  typedef sai_status_t (*sai_create_route_entry_fn)(
182          _In_ const sai_route_entry_t *route_entry,
183          _In_ uint32_t attr_count,
184          _In_ const sai_attribute_t *attr_list);
185
```

RouteはOIDを作らず(switch_id, vr_id, destination)
のタプルをOIDの代わりに使う
(経路は他のオブジェクトに比べて数が多くなるため)

```
144  /**
145   * @brief Unicast route entry
146   */
147  typedef struct _sai_route_entry_t
148  {
149      /**
150       * @brief Switch ID
151       *
152       * @objects SAI_OBJECT_TYPE_SWITCH
153       */
154      sai_object_id_t switch_id;
155
156      /**
157       * @brief Virtual Router ID
158       *
159       * @objects SAI_OBJECT_TYPE_VIRTUAL_ROUTER
160       */
161      sai_object_id_t vr_id;
162
163      /**
164       * @brief IP Prefix Destination
165       */
166      sai_ip_prefix_t destination;
167
168  } sai_route_entry_t;
```

```
    /**
     * @brief Next hop or next hop group id for the packet, or a router interface
     * in case of directly reachable route, or the CPU port in case of IP2ME route
     *
     * The next hop id is only effective when the packet action is one of the
     * following: FORWARD, COPY, LOG, TRANSIT.
     *
     * The next hop id can be a generic next hop object, such as next hop, next
     * hop group. Directly reachable routes are the IP subnets that are
     * directly attached to the router. For such routes, fill the router
     * interface id to which the subnet is attached. IP2ME route adds a local
     * router IP address. For such routes, fill the CPU port
     * (#SAI_SWITCH_ATTR_CPU_PORT).
     * When pointing to a next hop group which is empty, the effective routing
     * action will be DROP.
     *
     * When it is SAI_NULL_OBJECT_ID, then packet will be dropped.
     *
     * @type sai_object_id_t
     * @flags CREATE_AND_SET
     * @objects SAI_OBJECT_TYPE_NEXT_HOP, SAI_OBJECT_TYPE_NEXT_HOP_GROUP, SAI_OBJECT_TYPE_ROUTER_INTERFACE, SAI_OBJECT_TYPE_PORT
     * @allownull true
     * @default SAI_NULL_OBJECT_ID
     */
    SAI_ROUTE_ENTRY_ATTR_NEXT_HOP_ID,
```

nexthopをOIDで指定する
指定できるオブジェクトとして
- NEXT_HOP
- NEXT_HOP_GROUP
- ROUTER_INTERFACE
- PORT

が指定できる

右のどれにも該当しない普通のL3経路

connected経路

自分への経路
CPUポートを指定

ECMP/複数のnexthopを同時利用する場合

OSのネットワークスタックとの連携

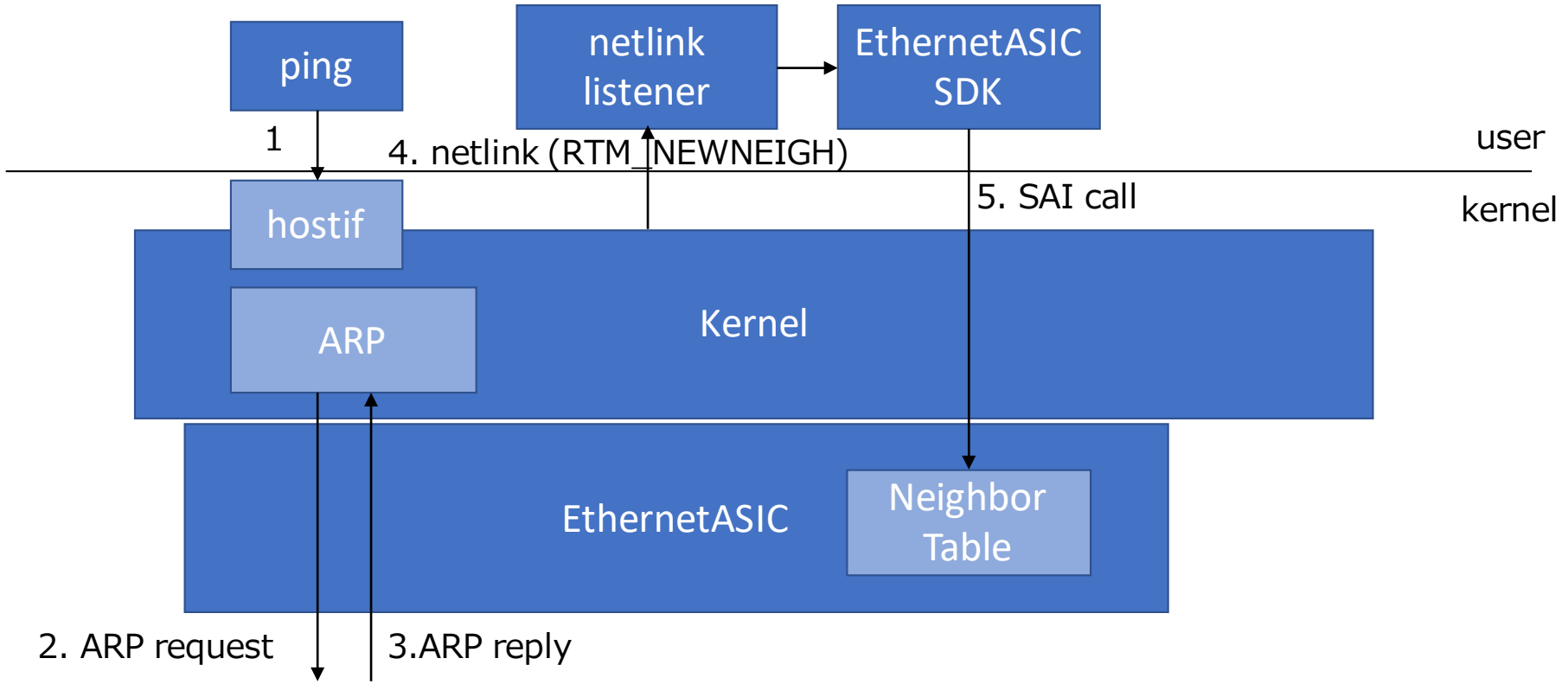
1. EthernetASICのフロントパネルポートに対応するネットワークインターフェイスの作成
   - PortとHost Interfaceオブジェクトを作成する

2. インターフェイスへのIPアドレスの設定
   - OSとEthernetASIC両者への操作が必要
   - RouterInterfaceとRouteオブジェクトを作成する

3. Neighborテーブル(ARP/ND)の同期
4. ルーティングプロトコルとの連携

3. Neighborテーブル(ARP/ND)の同期

- EthernetASICではARPの処理は行わず、OSのARP解決の結果をEthernetASICに反映する



Neithborテーブル(ARP/ND)の同期

EthernetASICに対して必要な操作

1. ARP/NDパケットのCPU側へのコピー
   - SONiCではCOPP(Control Plane Policing)の一環で設定される
   - 本日は詳細は割愛、概要を次に説明

2. Neighborテーブルにエントリーを追加

master

sonic-buildimage / dockers / docker-orchagent / copp.json.j2

COPPの設定ファイル

abdosi Changes to add template support for copp.json. (#5053) ...

1 contributor

85 lines (85 sloc) | 2.21 KB

```
[
    {
        "COPP_TABLE:default": {
            "queue": "0",
            "meter_type":"packets",
            "mode":"sr_tcm",
            "cir":"600",
            "cbs":"600",
            "red_action":"drop"
        },
        "OP": "SET"
    },
    {
        "COPP_TABLE:trap.group.bgp.lacp": {
            "trap_ids": "bgp,bgpv6,lacp",
            "trap_action":"trap",
            "trap_priority":"4",
            "queue": "4"
        },
        "OP": "SET"
    },
    {
        "COPP_TABLE:trap.group.arp": {
            "trap_ids": "arp_req,arp_resp,neigh_discovery",
            "trap_action":"copy",
            "trap_priority":"4",
            "queue": "4",
            "meter_type":"packets",
            "mode":"sr_tcm",
            "cir":"600",
            "cbs":"600",
            "red_action":"drop"
        },
        "OP": "SET"
    },
```

ARP_REQ/ARP_RESP/NDはCPUにコピーする設定
これらがSAIのhostif_table_entry, hostif_trap_group, hostif_trap, policierオブジェクトの作成に変換され、EthernetASICに設定される

https://github.com/Azure/sonic-buildimage/blob/master/dockers/docker-orchagent/copp.json.j2

NeighborEntryの作成

Routeは (switch_id, vr_id, destination(prefix))だった

NeighborEntryはOIDを作らず(switch_id, rif_id, ip_address)のタプルをOIDの代わりに使う
(NeighborEntryは他のオブジェクトより数が多くなるため)

```
259  /**
260   * @brief Neighbor table methods, retrieved via sai_api_query()
261   */
262  typedef struct _sai_neighbor_api_t
263  {
264      sai_create_neighbor_entry_fn         create_neighbor_entry;
265      sai_remove_neighbor_entry_fn         remove_neighbor_entry;
266      sai_set_neighbor_entry_attribute_fn  set_neighbor_entry_attribute;
267      sai_get_neighbor_entry_attribute_fn  get_neighbor_entry_attribute;
268      sai_remove_all_neighbor_entries_fn   remove_all_neighbor_entries;
269
270  } sai_neighbor_api_t;
```

```
195  /**
196   * @brief Create neighbor entry
197   *
198   * Note: IP address expected in Network Byte Order.
199   *
200   * @param[in] neighbor_entry Neighbor entry
201   * @param[in] attr_count Number of attributes
202   * @param[in] attr_list Array of attributes
203   *
204   * @return #SAI_STATUS_SUCCESS on success, failure status code on error
205   */
206  typedef sai_status_t (*sai_create_neighbor_entry_fn)(
207          _In_ const sai_neighbor_entry_t *neighbor_entry,
208          _In_ uint32_t attr_count,
209          _In_ const sai_attribute_t *attr_list);
210
```

```
169  /**
170   * @brief Neighbor entry
171   */
172  typedef struct _sai_neighbor_entry_t
173  {
174      /**
175       * @brief Switch ID
176       *
177       * @objects SAI_OBJECT_TYPE_SWITCH
178       */
179      sai_object_id_t switch_id;
180
181      /**
182       * @brief Router interface ID
183       *
184       * @objects SAI_OBJECT_TYPE_ROUTER_INTERFACE
185       */
186      sai_object_id_t rif_id;
187
188      /**
189       * @brief IP address
190       */
191      sai_ip_address_t ip_address;
192
193  } sai_neighbor_entry_t;
```

https://github.com/opencomputeproject/SAI/blob/master/inc/saineighbor.h#L264

NeighborEntryのアトリビュート

```
41  /**
42   * @brief Attribute Id for SAI neighbor object
43   */
44  typedef enum _sai_neighbor_entry_attr_t
45  {
46      /**
47       * @brief Start of attributes
48       */
49      SAI_NEIGHBOR_ENTRY_ATTR_START,
50
51      /**
52       * @brief Destination MAC address for the neighbor
53       * Valid only when SAI_NEIGHBOR_ENTRY_ATTR_IS_LOCAL == true
54       *
55       * @type sai_mac_t
56       * @flags MANDATORY_ON_CREATE | CREATE_AND_SET
57       */
58      SAI_NEIGHBOR_ENTRY_ATTR_DST_MAC_ADDRESS = SAI_NEIGHBOR_ENTRY_ATTR_START,
59
60      /**
61       * @brief L3 forwarding action for this neighbor
62       *
63       * @type sai_packet_action_t
64       * @flags CREATE_AND_SET
65       * @default SAI_PACKET_ACTION_FORWARD
66       */
67      SAI_NEIGHBOR_ENTRY_ATTR_PACKET_ACTION,
68
```

あるNeighbor(IPアドレス)に対応するMACアドレス
OSがARP/ND解決を行い、その結果をASICに格納する
(ASICはARP/ND処理はしない)

CREATE_AND_SETなので、後から対応するMACアドレスを変更することも可能

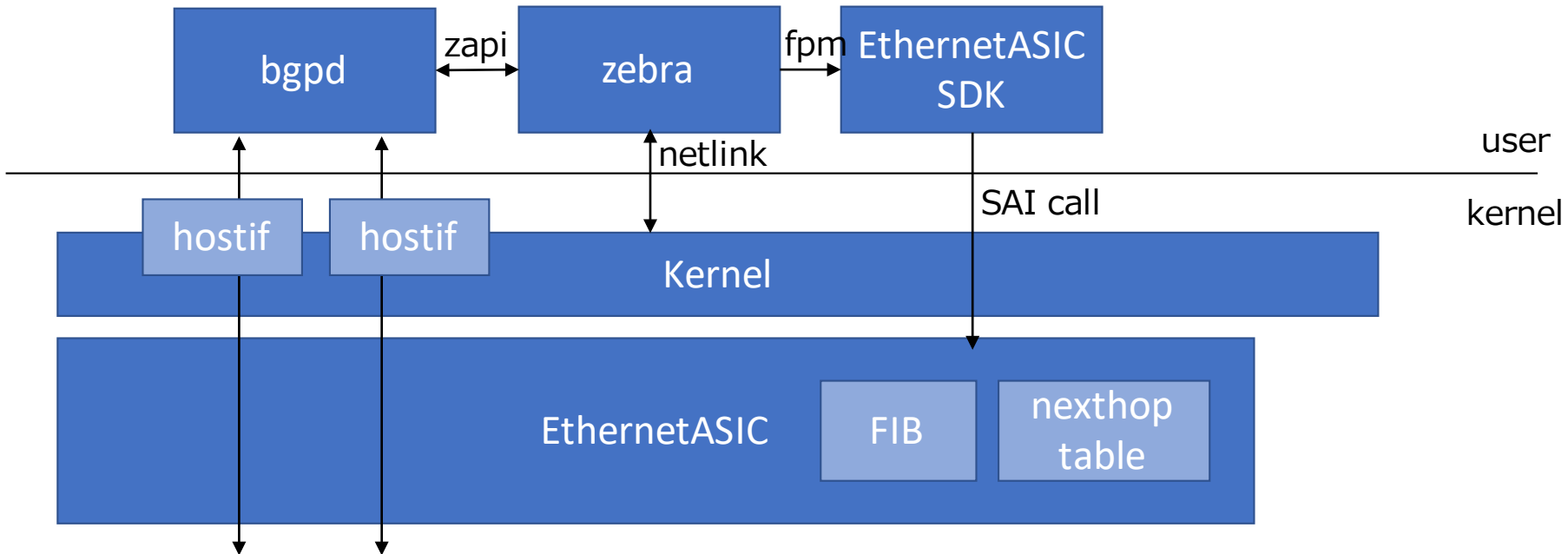
https://github.com/opencomputeproject/SAI/blob/master/inc/saineighbor.h#L58

OSのネットワークスタックとの連携

1. EthernetASICのフロントパネルポートに対応するネットワークインターフェイスの作成
   - PortとHost Interfaceオブジェクトを作成する
2. インターフェイスへのIPアドレスの設定
   - OSとEthernetASIC両者への操作が必要
   - RouterInterfaceとRouteオブジェクトを作成する
3. Neighborテーブル(ARP/ND)の同期
   - COPPを使ったARP/NDパケットのCPUへのコピー
   - OSにARP/ND処理を任せ、NeighborEntryオブジェクトを作成する
4. ルーティングプロトコルとの連携

4. ルーティングプロトコルとの連携

- SONiCではルーティングプロトコル実装としてFRR(Quaggaのfork)を利用
- FRRのFPM(Forwarding Plane Manager)という仕組みを利用し、FRRが経路をOSのFIBに注入する際に、その経路に関する通知をもらいEthernetASICに対して必要な操作を行う



ルーティングプロトコルとの連携

EthernetASICに対して必要な操作

1. Nexthopオブジェクトの作成
   - ECMPの場合はNexthopGroupを作りNexthopをグループ化(本日は割愛)

2. Routeオブジェクトの作成
   - “2. IPアドレス設定”でも作ったRouteオブジェクトだが、今回はNexthopにNexthopオブジェクトを設定する

```
363  /**
364   * @brief Next Hop methods table retrieved with sai_api_query()
365   */
366  typedef struct _sai_next_hop_api_t
367  {
368      sai_create_next_hop_fn          create_next_hop;
369      sai_remove_next_hop_fn          remove_next_hop;
370      sai_set_next_hop_attribute_fn   set_next_hop_attribute;
371      sai_get_next_hop_attribute_fn   get_next_hop_attribute;
372
373  } sai_next_hop_api_t;
```

```
309  /**
310   * @brief Create next hop
311   *
312   * Note: IP address expected in Network Byte Order.
313   *
314   * @param[out] next_hop_id Next hop id
315   * @param[in] switch_id Switch id
316   * @param[in] attr_count Number of attributes
317   * @param[in] attr_list Array of attributes
318   *
319   * @return #SAI_STATUS_SUCCESS on success, failure status code on error
320   */
321  typedef sai_status_t (*sai_create_next_hop_fn)(
322          _Out_ sai_object_id_t *next_hop_id,
323          _In_ sai_object_id_t switch_id,
324          _In_ uint32_t attr_count,
325          _In_ const sai_attribute_t *attr_list);
326
```

スイッチIDとアトリビュートのリストを受け取り、OIDを返す

Nexthopのアトリビュート

```
105 /**
106  * @brief Attribute id for next hop
107  */
108 typedef enum _sai_next_hop_attr_t
109 {
110     /**
111      * @brief Start of attributes
112      */
113     SAI_NEXT_HOP_ATTR_START,
114
115     /**
116      * @brief Next hop entry type
117      *
118      * @type sai_next_hop_type_t
119      * @flags MANDATORY_ON_CREATE | CREATE_ONLY
120      * @isresourcetype true
121      */
122     SAI_NEXT_HOP_ATTR_TYPE = SAI_NEXT_HOP_ATTR_START,
123
124     /**
125      * @brief Next hop entry IPv4 address
126      *
127      * @type sai_ip_address_t
128      * @flags MANDATORY_ON_CREATE | CREATE_ONLY
129      * @condition SAI_NEXT_HOP_ATTR_TYPE == SAI_NEXT_HOP_TYPE_IP or SAI_NEXT_HOP_ATTR_TYPE == SAI_NEXT_HOP_TYPE_MPLS
130      */
131     SAI_NEXT_HOP_ATTR_IP,
132
133     /**
134      * @brief Next hop entry router interface id
135      *
136      * @type sai_object_id_t
137      * @flags MANDATORY_ON_CREATE | CREATE_ONLY
138      * @objects SAI_OBJECT_TYPE_ROUTER_INTERFACE
139      * @condition SAI_NEXT_HOP_ATTR_TYPE == SAI_NEXT_HOP_TYPE_IP or SAI_NEXT_HOP_ATTR_TYPE == SAI_NEXT_HOP_TYPE_MPLS
140      */
141     SAI_NEXT_HOP_ATTR_ROUTER_INTERFACE_ID,
```

IPアドレスとRouterInterfaceのOIDでNexthopを表現する
どちらもCREATE_ONLYで変更不可。

NeighborEntryのキーは(switch_id, rif_id, ip_address)だったので、この2つの情報で
EthernetASICは使うべきMACアドレスを知ることができる

OSのネットワークスタックとの連携

1. EthernetASICのフロントパネルポートに対応するネットワークインターフェイスの作成
   - PortとHost Interfaceオブジェクトを作成する
2. インターフェイスへのIPアドレスの設定
   - OSとEthernetASIC両者への操作が必要
   - RouterInterfaceとRouteオブジェクトを作成する
3. Neighborテーブル(ARP/ND)の同期
   - COPPを使ったARP/NDパケットのCPUへのコピー
   - OSにARP/ND処理を任せ、NeighborEntryオブジェクトを作成する
4. ルーティングプロトコルとの連携
   - FRRのFPM機能を使って経路情報を得る
   - NexthopオブジェクトとRouteオブジェクトを作成する

まとめ

- ホワイトボックススイッチNOSではOSのネットワークスタックとEthernetASICをどのように連携させるかが肝

- SONiC/SAIによりオープンにNOSの内部実装に関する議論が出来るようになった

- 実際に手を動かして触れる実装があることで、より深くネットワーク機器を理解できるようになってきている